SET 24

PART A — Short questions and answers

Q1. Compare an ER diagram and a relational schema using LEASE as an example.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Interpret the statement: DELETE FROM LEASE WHERE LeaseEnd < '2024- 01-01';

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q3. Explain a determinant using Property_ID → Monthly_Rent.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q4. Explain the use of the IN operator to filter properties from several localities.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Describe the use of MIN(MonthlyRent) grouped by Locality.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q6. Explain the role of the WHEN OTHERS exception handler.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q7. Explain why relationship-centred queries are costly in a relational database.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q8. Describe the use of the limit() and skip() methods.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Describe the output of the MongoDB query: db.properties.find({Locality: "Adyar", MonthlyRent: {$lt: 20000}})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Interpret the Cypher query: MATCH (o:Owner)-[:OWNS]->(p:Property) RETURN o.name, p.type;

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: A property rental agency maintains a Property Rental Management System to store tenant details,

1(a) ER design:

• **TENANT**(TenantID, TenantName, Phone, Occupation); primary key: **TenantID**.

• **PROPERTY**(PropertyID, PropertyType, Locality, MonthlyRent, OwnerName); primary key: **PropertyID**.

• **LEASE**(LeaseID, TenantID, PropertyID, LeaseStart, LeaseEnd); primary key: **LeaseID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE TENANT (
  TenantID NUMBER PRIMARY KEY,
  TenantName VARCHAR2(100),
  Phone VARCHAR2(100),
  Occupation VARCHAR2(100)
```

```
);
CREATE TABLE PROPERTY (
  PropertyID NUMBER PRIMARY KEY,
  PropertyType VARCHAR2(100),
  Locality VARCHAR2(100),
  MonthlyRent VARCHAR2(100),
  OwnerName VARCHAR2(100)
);
CREATE TABLE LEASE (
  LeaseID NUMBER PRIMARY KEY,
  TenantID NUMBER,
  PropertyID NUMBER,
  LeaseStart VARCHAR2(100),
  LeaseEnd VARCHAR2(100)
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM LEASE x
JOIN TENANT a ON x.TenantID = a.TenantID
JOIN PROPERTY b ON x.PropertyID = b.PropertyID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. Graph database (Neo4j/Cypher): Create the required node labels and relationship types exactly as specified in the set. Example pattern:

```
CREATE (a:EntityA {id: 1, name: 'Name A'});
CREATE (b:EntityB {id: 2, name: 'Name B'});
MATCH (a:EntityA {id: 1}), (b:EntityB {id: 2})
CREATE (a)-[:RELATED_TO]->(b);
MATCH (a:EntityA)-[:RELATED_TO]->(b:EntityB)
RETURN a.name, b.name;
```